

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Programación 2

Trabajo Práctico Integrador

FOOD STORE

Sistema de Gestión de Pedidos de Comida

Alumno: Carlos Gabriel Chiavarini

Año: 2026

Índice

- 2. Marco Teórico
- 3. Decisiones Técnicas y Arquitectura
- 4. Dificultades y Soluciones
- 5. Bibliografía
- 6. Repositorio y Video

Marco Teórico

El presente trabajo práctico integrador fue desarrollado utilizando Java como lenguaje de programación y tomando como base los conceptos estudiados durante la cursada de Programación 2. El objetivo principal del proyecto consistió en construir una aplicación de consola capaz de gestionar categorías, productos, usuarios y pedidos dentro de un sistema de venta de comidas. A través de este desarrollo se buscó aplicar en un caso práctico los conceptos fundamentales de la Programación Orientada a Objetos y las buenas prácticas de diseño vistas durante la materia.

La Programación Orientada a Objetos propone modelar los problemas mediante entidades que poseen estado y comportamiento. Esta forma de organizar el software permite representar situaciones del mundo real de manera más natural y facilita la construcción de sistemas mantenibles a largo plazo. En este proyecto, cada una de las entidades del dominio fue modelada como una clase independiente, permitiendo que los datos y las operaciones relacionadas permanezcan agrupados dentro de una misma estructura.

Uno de los pilares aplicados fue el encapsulamiento. Los atributos de las clases fueron declarados como privados para evitar modificaciones externas no controladas. En lugar de exponer setters públicos para cada atributo, se optó por implementar métodos con significado de negocio que validan la información antes de modificar el estado interno de los objetos. Esta decisión contribuye a preservar la consistencia de los datos y reduce el riesgo de errores provocados por modificaciones indebidas.

Otro concepto importante utilizado fue la herencia. Todas las entidades principales heredan de una clase abstracta denominada Base, la cual concentra los atributos comunes del sistema, como el identificador único, la fecha de creación y el indicador de baja lógica. Gracias a esta estrategia se evitó la duplicación de código y se consiguió una estructura más uniforme y fácil de mantener.

También se emplearon interfaces para representar comportamientos compartidos. La interfaz Calculable fue implementada por la clase Pedido para establecer un mecanismo estandarizado de cálculo del total. Esta decisión permitió demostrar el uso de polimorfismo y separación de responsabilidades dentro del modelo de dominio.

La aplicación almacena toda la información en memoria utilizando colecciones dinámicas. El uso de ArrayList permitió gestionar conjuntos de objetos sin necesidad de recurrir a una base de datos. Aunque esta decisión implica que la información se pierde al finalizar la ejecución, resulta adecuada para los objetivos académicos planteados por la consigna y permite concentrar el trabajo en los conceptos de programación orientada a objetos.

Finalmente, se implementó un conjunto de excepciones personalizadas para representar errores de negocio específicos. Este enfoque permitió capturar situaciones inválidas, informar mensajes descriptivos al usuario y mantener la estabilidad de la aplicación durante toda su ejecución.

Decisiones Técnicas y Arquitectura

Durante el diseño de la solución se decidió adoptar una arquitectura organizada por capas. Aunque la consigna proporcionaba una estructura mínima, se incorporaron componentes adicionales con el objetivo de mejorar la claridad y mantenibilidad del código. Esta organización permitió distribuir las responsabilidades de forma más equilibrada y evitar que la lógica del sistema quedara concentrada en una única clase.

Las entidades fueron utilizadas para representar el dominio del problema. Cada una de ellas contiene los atributos y comportamientos necesarios para cumplir con sus responsabilidades específicas. De esta manera, un producto conoce cómo validar su propio stock, un pedido sabe calcular su total y una categoría administra la relación con sus productos asociados. Esta aproximación favorece un diseño más cercano a los principios de la orientación a objetos.

Se incorporó una capa de servicios encargada de coordinar los casos de uso del sistema. Las clases `CategoriaService`, `ProductoService`, `UsuarioService` y `PedidoService` centralizan las operaciones CRUD, las validaciones generales y la coordinación entre entidades. Gracias a esta decisión, la lógica de negocio quedó desacoplada de la interfaz de usuario y resulta más sencilla de mantener o extender en futuras versiones.

Otra decisión importante fue la creación de una clase utilitaria denominada `InputHelper`. Esta clase concentra la lectura y validación de datos ingresados por consola, evitando la repetición de código en los distintos menús del sistema. Además, permitió mejorar la experiencia de uso mediante mensajes consistentes y validaciones reutilizables.

Durante el desarrollo también se procuró aplicar el principio `Tell Don't Ask`. En lugar de consultar el estado de los objetos para luego modificarlo desde el exterior, se delegó la responsabilidad en las propias entidades. Por ejemplo, las operaciones relacionadas con el stock se ejecutan mediante métodos específicos de la clase `Producto` y no a través de modificaciones directas de atributos. Esta práctica favorece el encapsulamiento y reduce el acoplamiento entre clases.

Con respecto a la eliminación de registros, se optó por implementar bajas lógicas. Esta técnica consiste en marcar los objetos como eliminados sin retirarlos físicamente de las colecciones. De esta manera se conserva el historial de información y se evita afectar relaciones existentes entre las distintas entidades del sistema.

La arquitectura final logró mantener una separación clara entre la interacción con el usuario, la lógica de negocio y el modelo de dominio. Esta organización facilita la comprensión del código y representa una aproximación más cercana a la utilizada en aplicaciones reales.

Dificultades y Soluciones

Uno de los desafíos más importantes fue definir correctamente las relaciones entre las distintas entidades del sistema. La interacción entre categorías, productos, usuarios, pedidos y detalles de pedido exigió analizar cuidadosamente las responsabilidades de cada clase para evitar dependencias innecesarias y mantener la coherencia del modelo.

La implementación de los pedidos representó una de las partes más complejas del proyecto. Fue necesario garantizar que cada pedido pudiera contener múltiples detalles, calcular subtotales automáticamente y actualizar el total general de forma consistente. Además, debían respetarse restricciones relacionadas con el stock disponible y la validez de los productos seleccionados.

Otro aspecto que requirió especial atención fue la validación de los datos ingresados por el usuario. Debido a que la aplicación funciona íntegramente por consola, existe una alta probabilidad de recibir entradas inválidas. Para resolver este problema se incorporaron validaciones específicas y excepciones personalizadas capaces de detectar errores antes de que afectaran el funcionamiento del sistema.

También surgieron desafíos relacionados con el manejo de identificadores únicos para cada entidad. Se optó por utilizar contadores internos dentro de los servicios para generar los identificadores de forma automática. Esta solución permitió simplificar la creación de objetos y asegurar que no existieran duplicados dentro de las colecciones.

La aplicación del principio Tell Don't Ask también representó un aprendizaje importante. En una primera aproximación resultaba tentador exponer setters públicos y realizar modificaciones desde distintas partes del sistema. Sin embargo, se decidió encapsular la lógica dentro de las propias entidades mediante métodos específicos de negocio. Esta decisión implicó un mayor esfuerzo inicial, pero produjo un diseño más sólido y coherente.

Por último, fue necesario realizar diversas pruebas funcionales para verificar que todas las historias de usuario contempladas por la consigna se comportaran correctamente. Estas pruebas permitieron detectar errores de validación, ajustar la gestión de excepciones y mejorar la estabilidad general del sistema.

Bibliografía

Para la realización de este trabajo se consultó la documentación oficial de Java publicada por Oracle, especialmente la relacionada con colecciones, excepciones, clases utilitarias y programación orientada a objetos. También se utilizó el material teórico y práctico proporcionado por la cátedra de Programación 2 durante el desarrollo de la materia.

Adicionalmente se revisaron recursos educativos vinculados con principios de diseño orientado a objetos, encapsulamiento, herencia, interfaces y buenas prácticas de programación. Estas fuentes sirvieron como apoyo para fundamentar decisiones de diseño y resolver problemas surgidos durante la implementación del proyecto.

Repositorio y Video

Enlace al video en YouTube: <https://youtu.be/b092i7t7B50>

Enlace al repositorio en GitHub:

<https://github.com/cgchiavarini/UTN-TUPAD-TPI-PROGRAMACION-2.git>